Wiring JEPA for Physical Simulations: Masking, Targets, and Parameter Recovery on Active Matter

Jubin Pakdel [1]

Abstract

Representation learning of spatiotemporal physical systems borrows from computer vision, where a model predicts the masked part of an image or video from the context. Unlike images and video, these systems are governed by known equations, which raises the question of how such a prior can improve the representations a model learns. In an active matter simulation, differentiating the orientation and velocity fields makes the active dipole strength α a linear coefficient between them. A model could learn that coefficient by predicting one field from the other, or by predicting both from a partial context, so I ablate which fields serve as context and target, whether they are differentiated, and how much of the context is hidden.

Following Qu et al. (2026), I train every task with both a predictor in latent space (JEPA) and a decoder in pixel space (MAE). Unlike the previous study, in which JEPA predicts the next window and MAE reconstructs the same one, I give both models the same prediction task, so each task can be compared in latent and pixel space. Across the 48 ablations, the 20 best all come from differentiated inputs, but the improvement comes from recovering the steric alignment ζ, not α. JEPA achieves a lower average MSE than MAE on 22 of the 24 matched ablations. Although the best MAE comes within 0.6 points of the best JEPA (2.65% against 2.03%), MAE's worst case is 3× JEPA's (31.21% against 9.80%). Representations formed by latent prediction may vary less with the choice of context, target and mask than those formed by pixel prediction.

[1]Courant Institute of Mathematical Sciences, New York University, New York, NY, USA. Correspondence to: Jubin Pakdel <jjp9216@nyu.edu>.

Preprint. October 26, 2026.

1. Introduction

Active matter is a fluid of self-propelled particles that generate the flow around them. The Well's `active_matter` simulation (Ohana et al., 2024) records this system as a set of distinct physical fields, among them the concentration, the velocity u, the orientation D, and the strain rate. Each run is generated under two scalar parameters, the active dipole strength α and the steric alignment ζ. The orientation drives the velocity through the Stokes equation. Keeping only its active term,

$$\Delta u = -\alpha\,(\nabla \cdot D) + \nabla\Pi, \qquad \nabla \cdot u = 0,$$

where Π is the pressure field maintaining incompressibility. The Stokes operator is nonlocal, so the velocity at a point depends on the orientation everywhere in the box, while its Laplacian at that point depends on the divergence of the orientation there. Differentiating the two fields therefore turns the coupling into a relationship at each grid point, with α as the coefficient between two channels. Does an encoder that sees $\nabla \cdot D$ and Δu recover α more easily than one that sees D and u?

A Joint Embedding Predictive Architecture (JEPA) splits a single input into two parts, x and y, and learns to predict the latent representation of one part from the other, forcing their embeddings to agree in representation space. JEPA's foundational papers (Assran et al., 2023; Bardes et al., 2024) set x and y as two crops of the same image or two sets of spatiotemporal patches of a video.

Qu et al. (2026) ask whether latent prediction learns more meaningful representations of a physical system than pixel prediction, comparing JEPA against a masked autoencoder (MAE) (He et al., 2022) on three physical systems, including active matter. They find that JEPA, trained to predict the next window of each channel from the current one, outperforms MAE, which instead reconstructs the masked part of the same window. Informed by the physics based prior, I select different fields as context and target to see how changing the learning objective changes the outcome, and how their result holds when both models share it. I ablate different masking strategies and regularizer weights to serve as control for the ablations. Every prediction task is trained with both a predictor in latent space (JEPA) and a decoder in pixel space

(MAE). Together these give four prediction tasks at three masking levels and two regularizer weights, 48 ablations across the two models.

21 of the top 24 ablations use differentiated inputs. Counter to the hypothesis, differentiation improves ζ recovery by 2–5× and not α. On identical next-window prediction tasks, JEPA outperforms MAE in 22 of 24. Although both see the same tasks and masks, MAE's error varies far more across ablations, and its worst case is 3× JEPA's.

2. Physics and the Hypothesis

2.1. The Active Matter System and Its Parameters

I work with The Well's `active_matter` simulation (Ohana et al., 2024), a 2D nematic active fluid governed by a kinetic theory of rod-like particles immersed in a Stokes flow. Each trajectory is generated under two scalar parameters, the active dipole strength α (5 levels) and the steric alignment ζ (9 levels), 45 combinations in all. The state is an 11-channel field decomposing as concentration $\phi \in \mathbb{R}$, velocity $u \in \mathbb{R}^2$, orientation tensor $D \in \mathbb{R}^{2\times2}$, and strain rate $E \in \mathbb{R}^{2\times2}$, all on a 256×256 grid sampled at T=16 frames per window.

The governing equations operate sequentially, producing the channels in a causal order,

$$\psi \xrightarrow{\text{moments}} (\phi,\, D) \xrightarrow{\text{Stokes eq.}} u \xrightarrow{\text{spatial deriv.}} E. \qquad (1)$$

The simulation evolves a kinetic distribution ψ. The moments of ψ yield the concentration ϕ and the orientation D, and the remaining channels follow from D.

The orientation drives the velocity through the Stokes equation. The flow is forced by the divergence of a stress built from D. Because the Stokes operator is linear, the stress determines the velocity, and the stress carries both parameters as coefficients. Its active term enters with coefficient α, and its steric term, a nonlinear function of D, enters with a coefficient proportional to ζ (Maddu et al., 2024).

D and u are therefore sufficient to recover both parameters. The two parameters enter the equations differently. α appears in one place, as the coefficient of the active stress, and reaches every channel through the flow it forces. ζ appears in two, the steric stress and the alignment of the particles themselves, so it acts on D directly as well as through the flow. This difference will matter throughout, since a design choice that helps one parameter may not help the other.

The differentiated fields $\nabla \cdot D$ and Δu turn the same coupling into a relationship between the fields at each grid point. The differentiation is invertible on this data, u is recoverable from Δu up to its box average and the orientational content of D from $\nabla \cdot D$, so the differentiated pair carries the same information as the raw one. It reweights every spatial mode by its wavenumber, relocating the fields' energy from low frequencies toward high ones.

2.2. The Hypothesis

To learn α from D and u, an encoder has to undo the Stokes solve. With $\nabla \cdot D$ and Δu, α becomes the coefficient between the two channels at each point, up to the pressure gradient and the steric stress. The hypothesis is that an encoder that sees the differentiated pair recovers α more easily than one that sees the raw pair. The same equation carries ζ as the coefficient of the steric stress, but that stress is a nonlinear function of D, the raw input.

2.3. Questions

The hypothesis and the result of Qu et al. (2026) raise two questions that set the experiment design. Does the differentiated pair recover α better than the raw pair, and does that answer change between pixel and latent prediction or when the input is masked? The tasks of Section 3 vary the fields the encoder sees, the model, and the mask.

3. Experiment Setup

3.1. Baseline

The prediction task chooses the two parts x and y, with x entering the context branch and y the target. The context encoder f_θ maps x to N=2048 tokens, $\mathbf{z}_x = f_\theta(x) \in \mathbb{R}^{N\times d}$, and the target branch encodes y to $\mathbf{z}_y$ in the same way. A shallow transformer predictor s_ϕ maps $\mathbf{z}_x$ to a prediction of $\mathbf{z}_y$. Training minimizes their disagreement in representation space, with a regularizer R applied to the context embedding,

$$\mathcal{L}(\mathbf{z}_x, \mathbf{z}_y) \;=\; \underbrace{\|s_\phi(\mathbf{z}_x) - \mathbf{z}_y\|_2^2}_{\text{predictor MSE}} + \lambda \cdot R(\mathbf{z}_x). \qquad (2)$$

The encoder is a ViT-small, and the target branch is an exponential moving average (EMA) copy of the context encoder with its gradients stopped. R is VICReg (Bardes et al., 2022). Its variance term keeps each dimension's spread above a threshold, so the encoder cannot produce the same output for every input, and its covariance term pushes the dimensions to store different information. These choices are held fixed, and every run uses $\lambda \in \{0.01, 0.1\}$. Each task chooses which channels enter each branch, whether the prediction is made in latent space by a predictor or in pixel space by a decoder, and whether the input is masked.

The baseline sets x and y to adjacent windows of T=16 frames from the same trajectory, all 11 channels on both branches, the standard spatiotemporal JEPA recipe (Bardes

et al., 2024) applied to physics data. As a distribution it models

$$p(\phi_{t+1}, u_{t+1}, D_{t+1}, E_{t+1} \mid \phi_t, u_t, D_t, E_t),$$

the next window of the full coupled state given the current one.

3.2. The Input

The remaining channels follow from D and u. The concentration is the trace of D and the strain rate is the symmetric gradient of u, so the two fields carry the whole state. Since this paper studies the simple relationship between them, every task uses only these two fields, raw or differentiated.

A model can learn this relationship by modeling the joint distribution of the two fields, seeing both, or the conditional distribution of one given the other, seeing only one.

The tasks `du_du` and `dl_dl` see both fields, raw and differentiated respectively, and predict their next window like the baseline,

$$p(u_{t+1}, D_{t+1} \mid u_t, D_t),$$
$$p(\Delta u_{t+1}, \nabla \cdot D_{t+1} \mid \Delta u_t, \nabla \cdot D_t).$$

The tasks `u_multi` and `lapu_multi` see only the flow field, u or its Laplacian Δu respectively. Each is trained to predict its own next window along with the other field's current and next window,

$$\texttt{u_multi}\colon\ p(D_t, D_{t+1}, u_{t+1} \mid u_t),$$
$$\texttt{lapu_multi}\colon\ p(\nabla \cdot D_t, \nabla \cdot D_{t+1}, \Delta u_{t+1} \mid \Delta u_t).$$

3.3. JEPA and MAE

Every task is learned twice, once by JEPA with the baseline recipe above and once by MAE, which replaces the target branch with a decoder, the same shallow transformer as the predictor followed by one linear layer that maps each token back to its patch of pixels. The prediction term of Equation (2) becomes the mean squared error between the decoded patches and the target in pixel space. Everything else is identical, so the two models learn the same prediction task, one in latent space and one in pixel space.

This differs from the MAE of Qu et al. (2026), which hides and predicts within the same temporal window. Here the hidden patches are filled with their mean value and still passed to the encoder rather than dropped, the loss covers every pixel rather than the masked ones only, and the target is the next window. The masks hide up to 60% of each field, and MAE's prediction loss carries the same VICReg penalty, to control for the possibility that the regularizer, and not the prediction task, is what produces better representations. In tests of its impact on MAE for the two pair tasks, the penalty stabilized the probe results, making them less variant to the prediction task. It improved them by a large margin with a full context and changed them little with masking, except `du_du` with the weak mask and λ=0.1, which it worsened by 5.8 points.

3.4. Masking

Each task is trained at three levels of input masking. In the first, the encoder sees its full context and predicts the next window. In the other two, the context is masked, meaning the encoder sees a partial context, in which part of each context channel in the current window is hidden before it enters the encoder. The unmasked current window is added to the target, so the encoder must reconstruct what it did not see as well as predict what comes next. I run this at two strengths, a strong and a weak mask that hide up to 60% and 25% of each context channel, so the encoder sees between 40% and 80% of its input. Every task hides part of the data from the encoder, since the target is always a window or a channel it does not see, so in a broad sense every task is masked prediction, the objective MAE is built on. When this paper mentions masking in the context of ablations, it refers to the partial context described here.

3.5. Protocol

I compute the differentiated fields from the stored fields with central difference kernels, and their values are one to two orders of magnitude smaller than those of the raw fields. Without normalization, the encoder trained on the differentiated pair collapsed, and its probe results were no better than predicting the mean. I therefore normalize each differentiated channel with statistics from the training split. To check that the gain comes from the differentiation and not from the normalization alone, I normalized the raw pair and the baseline's channels the same way. Normalizing the raw pair barely changed its ζ recovery, from 8.66% to 8.34%, against 2.55% for the normalized differentiated pair, and worsened α recovery for both. Every run trains for 10,500 steps at batch size 2. The probe reads the mean pooled context tokens of the current window on a 700/96/104 train/validation/test split of windows. It fits a linear probe and a kNN regressor, choosing the regularization strength, k and the distance on the validation split. It reports test mean squared error on z-scored α and ζ as a percentage of the target's variance, so predicting the mean scores 100%. Every error below is on that scale. JEPA runs use five seeds and MAE runs use two. Table 1 reports the mean and standard deviation over seeds, and the other tables report means. The baseline runs, the λ=0 MAE runs, the normalization controls and a second set of five seeds of the raw and differentiated pairs used a local RTX 4070 SUPER, and every other run used cloud hosted RTX 3090s. The raw and differentiated pairs ran at five seeds on both

Table 1. Test error (%) on z-scored parameters for JEPA with a full context, mean and standard deviation over five seeds. Baseline rows ran on the local GPU and the rest on the cloud hosted GPUs.

task	λ	α lin	ζ lin	α kNN	ζ kNN
baseline	0.01	1.00 ± 0.22	6.80 ± 0.39	4.37 ± 2.37	11.67 ± 1.43
baseline	0.1	5.07 ± 1.61	14.23 ± 2.34	14.07 ± 8.58	19.84 ± 5.97
`du_du`	0.01	0.72 ± 0.15	8.00 ± 1.02	0.52 ± 0.17	10.48 ± 1.97
`du_du`	0.1	2.79 ± 1.67	12.54 ± 1.93	4.00 ± 4.89	16.66 ± 1.83
`dl_dl`	0.01	1.24 ± 0.14	3.35 ± 0.72	1.69 ± 0.69	6.43 ± 0.83
`dl_dl`	0.1	1.43 ± 0.34	2.53 ± 0.62	0.89 ± 0.43	3.25 ± 1.26
`u_multi`	0.01	1.35 ± 0.14	7.12 ± 1.05	3.46 ± 0.44	9.65 ± 0.92
`u_multi`	0.1	2.84 ± 0.34	7.41 ± 0.92	4.55 ± 1.02	10.26 ± 0.48
`lapu_multi`	0.01	1.50 ± 0.16	2.16 ± 0.25	2.77 ± 0.29	5.03 ± 0.47
`lapu_multi`	0.1	2.96 ± 0.65	3.76 ± 0.50	5.03 ± 1.02	4.12 ± 0.63

machines, and the ranges over seeds overlap in all sixteen comparisons.

4. Observations

4.1. The Input

Table 1 varies the input while holding the model fixed at the baseline recipe, JEPA with a full context. The differentiated pair's ζ probes beat the raw pair's at each regularizer weight, by 2.4× and 5.0× on the linear probe, and the differentiated flow field's linear ζ probe beats the raw one's by 3.3× at λ=0.01. Neither pair has better α recovery at both regularizer weights, and the best α probe in the table is the raw pair's kNN at 0.52%, roughly 8× lower than the baseline's. Counter to the hypothesis, differentiating the inputs improves ζ recovery and not α.

Does the ζ probe's improvement depend on the model? With MAE the differentiated pair's ζ error is 3.23% against the raw pair's 15.41% at λ=0.01 (Table 2), so the improvement holds for both models.

4.2. JEPA and MAE

Table 2 reports every task for both models at all three masking levels. Averaging all four probes, JEPA has the lower error in 22 of the 24 matched ablations (Figure 1). When provided a full context, MAE has a higher α error rate than JEPA in every row, by up to 6.6× on the linear probe (4.70% against 0.72% for `du_du` at λ=0.01). Its error rate on ζ is higher in every raw field ablation, by 1.9–4× (29.68% against 7.41% for `u_multi` at λ=0.1), and the two tie on the differentiated pair at λ=0.01 (3.23% against 3.35%). With masking, MAE's ζ probe comes within 1.8× of JEPA's (13.48% against 7.43% for `du_du` with the strong mask and λ=0.01), and the two tie on the differentiated pair at λ=0.01 (2.71% each). Across the ablations, MAE's average error ranges from 2.65% to 31.21%, against 2.03% to 9.80% for JEPA.

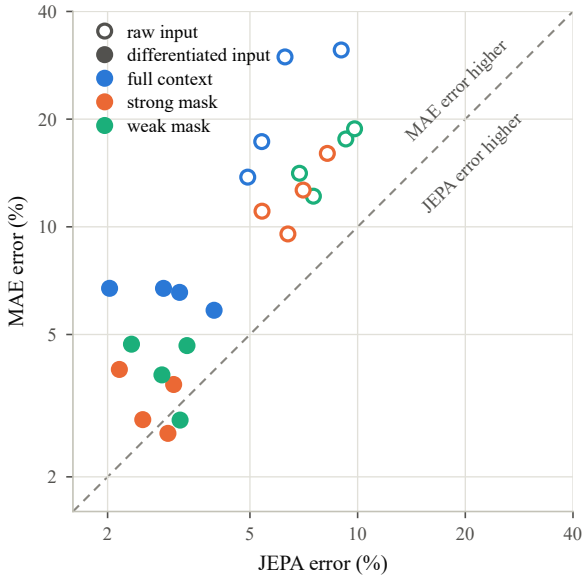


Figure 1. Error of JEPA against MAE on each of the 24 matched ablations, averaged over all four probes and seeds (Table 2). Color marks the masking level, and filled points are differentiated inputs. Points above the dashed line are ablations where MAE has the higher error.

4.3. Masking

Masking the context improves MAE in every ablation, whereas for JEPA it does not guarantee improvement and changes the error rate much less. Averaging the two linear probes, the strong mask lowers MAE's error in all eight task and regularizer weight combinations, by 1.1 to 12.2 points, and moves JEPA's by at most 3.0 points. With the regularizer turned off, masking takes MAE's error on the raw pair from 44.4% to 7.7% with the weak mask, and on

Table 2. Linear probe test error (%) on z-scored parameters, and the mean of all four probes, for each task, model and masking level, mean over seeds (five for JEPA, two for MAE). The strong and weak masks hide up to 60% and 25% of each context channel. All runs used the cloud hosted GPUs.

			full context			strong mask			weak mask		
task	λ	model	α lin	ζ lin	mean	α lin	ζ lin	mean	α lin	ζ lin	mean
du_du	0.01	JEPA	0.72	8.00	4.93	0.94	7.43	5.40	1.81	10.12	7.52
		MAE	4.70	15.41	13.77	2.07	13.48	11.06	2.39	14.25	12.18
du_du	0.1	JEPA	2.79	12.54	9.00	1.66	7.59	7.04	2.85	12.09	9.27
		MAE	6.45	34.47	31.21	2.65	13.85	12.67	2.83	20.38	17.60
dl_dl	0.01	JEPA	1.24	3.35	3.18	0.74	2.71	3.06	0.86	2.83	2.84
		MAE	3.68	3.23	6.56	1.33	2.71	3.62	1.87	3.52	3.86
dl_dl	0.1	JEPA	1.43	2.53	2.03	2.04	1.74	2.51	2.23	2.52	3.33
		MAE	4.79	8.67	6.73	1.73	2.27	2.89	2.98	2.98	4.66
u_multi	0.01	JEPA	1.35	7.12	5.39	0.69	10.46	6.38	1.36	10.96	6.88
		MAE	2.16	20.36	17.31	2.46	12.49	9.55	2.13	18.42	14.12
u_multi	0.1	JEPA	2.84	7.41	6.27	2.55	12.83	8.23	2.93	14.38	9.80
		MAE	3.34	29.68	29.85	2.62	20.58	16.04	2.86*	23.07*	18.80*
lapu_multi	0.01	JEPA	1.50	2.16	2.87	1.36	1.82	2.16	1.48	1.94	2.33
		MAE	3.88	3.36	6.73	2.58	2.41	3.99	2.55	3.23	4.70
lapu_multi	0.1	JEPA	2.96	3.76	3.97	2.82	1.82	2.95	3.24	2.09	3.19
		MAE	4.43	5.07	5.84	1.95	2.70	2.65	2.95	2.21	2.88

* one seed, the second never completed.

the differentiated pair from 15.5% to 2.1% with the strong mask.

For JEPA, the effect of masking depends on which field the encoder sees. It improves ζ recovery by a factor of two for the differentiated flow field at λ=0.1 (3.76% to 1.82% linear, 4.12% to 2.19% kNN) but worsens it for the raw flow field at every masking level and regularizer weight (7.12% to 10.46% linear with the strong mask and λ=0.01). MAE's ζ probe on the raw flow field improves with masking (20.36% to 12.49% at λ=0.01), so on that field the two models move in opposite directions. On the differentiated pair at λ=0.1, masking improves ζ recovery at the cost of α, lowering its linear ζ error from 2.53% to 1.74% while raising its α kNN error from 0.89% to 3.25%.

5. Explanation

5.1. Band Analysis

For every training window I split each raw field into eight feature bands, from wider than 128 pixels down to 1 to 2 pixels, and measure how much of each band's variation across windows is due to ζ and α. As a control, I repeat the measurement 300 times with randomly assigned parameters, treating the top 5% of those results as a noise floor. In the velocity, 91% of the 2–4 pixel band's variation is due to ζ, though that band holds only 5.8×10^{-8} of the energy. In the 32–64 pixel band, 63–88% of the variation is due to α after each run's first window, while no band of the orientation field rises above the noise floor for α. The bands tied to ζ hold one to seven orders of magnitude less energy than those tied to α.

5.2. Differentiation as a High Pass Filter

A divergence multiplies each Fourier mode by its wavenumber and a Laplacian by its square, so both act as high pass filters. In each run's first window, the raw orientation field holds 68% of its energy in features wider than 128 pixels, and its divergence 91% in features 8 to 64 pixels wide. Rescaling a band does not change how much of its variation is due to each parameter, and the per band fractions of u and Δu differ by at most 0.05, so differentiation changes where the energy is without adding information. Although α becomes a coefficient between two channels, the learning task spends its capacity on the directions that dominate the input's variance (Balestriero & LeCun, 2024), and differentiation moved that variance onto finer bands, where a larger share of the variation comes from ζ.

6. Limitations

MAE was trained with two seeds and JEPA with five, and MAE uses the settings chosen for JEPA, including the training length, so a separately tuned MAE could close part of the difference. The masked runs of both models also add the unmasked current window to the target, and no run separates the mask from that added target. A run's probe error shifts by a median of 14% of its value, and by as much as 66%,

between the two GPU types, although the raw and differentiated pairs, which ran on both, overlap in their ranges over five seeds. The probes are linear and kNN on mean pooled features, so information they cannot read may still be in the embedding. Every run uses one simulation and one regularizer, so the results may not carry over to other physical systems or regularizers.

7. Conclusion

Fixing the prediction task and changing only whether it is solved in latent or pixel space shows that latent prediction learns some mappings that pixel prediction does not. When provided a full context, MAE has a higher average error than JEPA in every row, and it only comes close once masking adds the current window to its target, the reconstruction task it was designed for. MAE can match JEPA, so the advantage of latent prediction is not only that it discards detail it cannot predict, the reason usually given for it (Assran et al., 2023). It also makes a model more flexible in the choice of prediction task, allowing mappings that are further from reconstructing the input.

JEPA (Assran et al., 2023; Bardes et al., 2024) and other latent methods still build their tasks from two views or two masks of the same input, a design inherited from the masked reconstruction methods they set out to improve on. Future work should move the prediction task further from reconstruction, toward less trivial mappings, to see what a representation can learn when its task is not a copy of its input.

Compute Accounting

The JEPA grid in Tables 1 and 2 is 120 runs of 10,500 steps each on cloud hosted RTX 3090s, roughly 100 GPU-hours including evaluation. The 47 MAE runs took about 7.5 hours across cloud hosted RTX 3090s and cost $9.17. The ten baseline runs, the eleven MAE runs without the regularizer and the two normalization controls ran locally on an RTX 4070 SUPER, at about 51 minutes per run. The band analysis and the probes run on a CPU from cached features.

References

Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., and Ballas, N. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv:2301.08243.

Balestriero, R. and LeCun, Y. How learning by reconstruction produces uninformative features for perception. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pp. 2566–2585, 2024. arXiv:2402.11337.

Bardes, A., Ponce, J., and LeCun, Y. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2105.04906.

Bardes, A., Garrido, Q., Ponce, J., Chen, X., Rabbat, M., LeCun, Y., Assran, M., and Ballas, N. Revisiting feature prediction for learning visual representations from video, 2024. arXiv:2404.08471.

He, K., Chen, X., Xie, S., Li, Y., Dollár, P., and Girshick, R. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. arXiv:2111.06377.

Maddu, S., Weady, S., and Shelley, M. J. Learning fast, accurate, and stable closures of a kinetic theory of an active fluid. *Journal of Computational Physics*, 504: 112869, 2024. arXiv:2308.06675.

Ohana, R., McCabe, M., Meyer, L., Morel, R., Agocs, F. J., Beneitez, M., Berger, M., Burkhart, B., Burns, K., Dalziel, S. B., Fielding, D. B., Fortunato, D., Goldberg, J. A., Hirashima, K., Jiang, Y.-F., Kerswell, R. R., Maddu, S., Miller, J., Mukhopadhyay, P., Nixon, S. S., Shen, J., Watteaux, R., Régaldo-Saint Blancard, B., Rozet, F., Parker, L. H., Cranmer, M., and Ho, S. The Well: a large-scale collection of diverse physics simulations for machine learning. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2412.00568.

Qu, H., Morel, R., McCabe, M., Bietti, A., Lanusse, F., Ho, S., and LeCun, Y. Representation learning for spatiotemporal physical systems. In *ICLR Workshop on AI & PDE*, 2026. arXiv:2603.13227. Code: https://github.com/helenqu/physical-representation-learning.